

AIML SESSION 06 ASSIGNMENT SUBMISSION

Name of Student : Vaibhav Vijay Kate

Domain/Subject : AIML (Artificial Intelligence & Machine Learning)

College : Government Polytechnic College Washim

Branch : Information Technology

Session : 06

Q1

Q1. Creating Lists)

Write a program to create the following lists using different methods:

- a) A list of 5 integers directly.
- b) An empty list using both [] and list().
- c) A list containing mixed data types (int, string, float, boolean).
- d) A list of 7 zeros using repetition (*).

Print all lists with clear labels and add comments explaining each creation method.

Program Code:

```
# A list of 5 integers directly.
```

```
l1=[1,2,3,4,5]
```

```
print("List Of 5 Integers Directly:",l1)
```

```
# An empty list using both [] and list().
```

```
l2=[]
```

```
print("empty list using [] :",l2)
```

```
l3=list()
```

```
print("empty list using () :",l3)
```

```
# A list containing mixed data types (int, string, float, boolean).
```

```
l4=[1,"Breking bad",2.1,True]
```

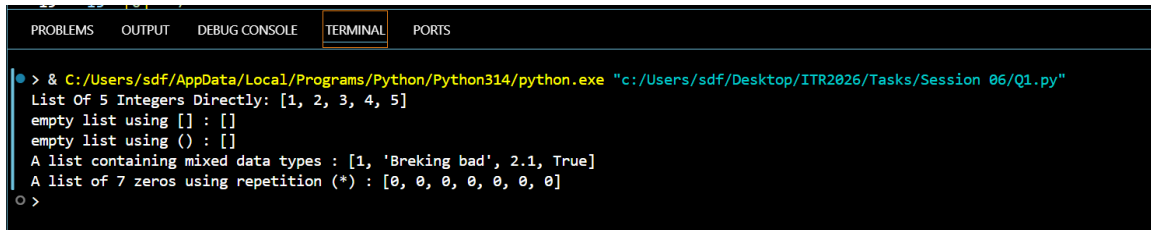
```
print("A list containing mixed data types :",l4)
```

```
# A list of 7 zeros using repetition (*).
```

```
l5= [0] * 7
```

```
print("A list of 7 zeros using repetition (*) :",l5)
```

Output Screenshot:



A screenshot of a terminal window with a dark background. The terminal has tabs at the top labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active), and 'PORTS'. The output shows the execution of a Python script. The first line is a command prompt followed by the command to run python.exe on a specific file. The output consists of several lines: a list of integers, two empty lists created using different methods, a list with mixed data types, and a list of seven zeros created using repetition. The prompt ends with a cursor.

```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 06/Q1.py"
List Of 5 Integers Directly: [1, 2, 3, 4, 5]
empty list using [] : []
empty list using () : []
A list containing mixed data types : [1, 'Breking bad', 2.1, True]
A list of 7 zeros using repetition (*) : [0, 0, 0, 0, 0, 0, 0]
>
```

Q2

Q2. Indexing and Negative Indexing)

Create a list: fruits = ["apple", "banana", "mango", "orange", "grape"]

Write code to print:

First item

Third item

Last item (using negative index)

Second last item (using negative index)

Take a number as input from the user and print the item at that index (handle basic cases only).

Program Code:

```
#Creating a List
```

```
fruits = ["apple", "banana", "mango", "orange", "grape"]
```

```
# Access first items
```

```
print("first item :",fruits[0])
```

```
# Access Third items
```

```
print("Third item :",fruits[2])
```

```
# Access Last item (using negative index)
```

```
print("Last item (using negative index) :",fruits[-1])
```

```
# Access Second last item (using negative index)
```

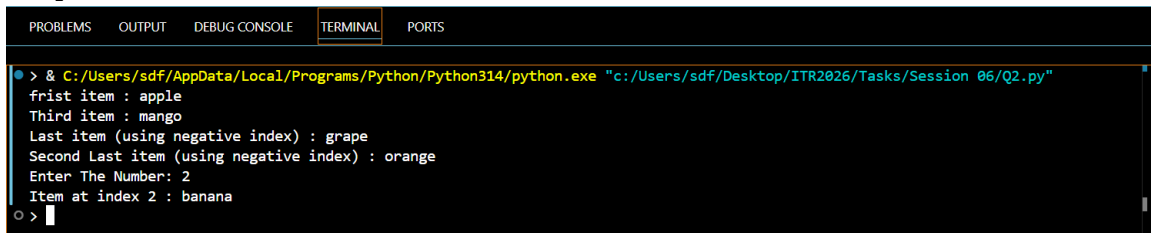
```
print("Second Last item (using negative index) :",fruits[-2])
```

```
# Take a number as input from the user and print the item at that index (handle basic cases only).
```

```
num = int(input("Enter The Number: "))
```

```
print("Item at index", num, ":", fruits[num-1])
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 06/Q2.py"
frist item : apple
Third item : mango
Last item (using negative index) : grape
Second Last item (using negative index) : orange
Enter The Number: 2
Item at index 2 : banana
>
```

Q3

Q3. Slicing Lists)

Given the list: numbers = 10, 20, 30, 40, 50, 60, 70, 80, 90, 100

Using slicing, print:

- a) First 4 elements
- b) Last 3 elements
- c) Elements from index 2 to 7
- d) Every second element
- e) The entire list in reverse order

Add comments explaining slicing syntax.

Session 06 (AIML) - Assignment Questions 1

Program Code:

```
numbers = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

```
# First 4 elements
```

```
print("First 4 elements :",numbers[:4:1])
```

```
# Last 3 elements
```

```
print("Last 3 elements :",numbers[10:6:-1])
```

```
# Elements from index 2 to 7
```

```
print("Elements from index 2 to 7 :",numbers[1:7:1])
```

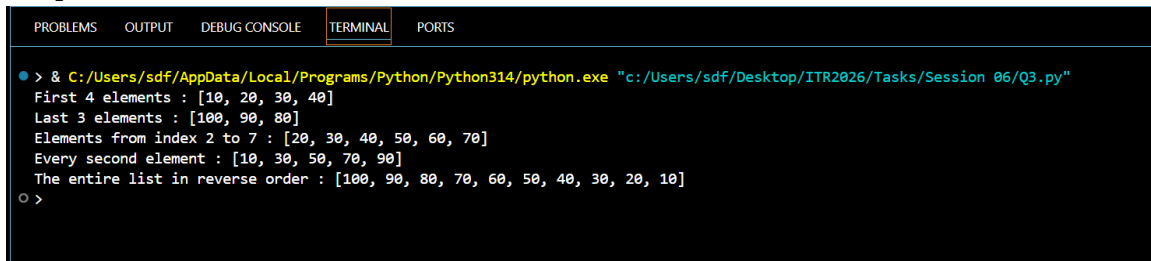
```
# Every second element
```

```
print("Every second element :",numbers[::2])
```

```
# The entire list in reverse order
```

```
print("The entire list in reverse order :",numbers[::-1])
```

Output Screenshot:



The screenshot shows a VS Code interface with a terminal window open. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (which is active), and PORTS. The terminal content shows a command to run a Python script, followed by several lines of output displaying list slices and a reversed list.

```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 06/Q3.py"
First 4 elements : [10, 20, 30, 40]
Last 3 elements : [100, 90, 80]
Elements from index 2 to 7 : [20, 30, 40, 50, 60, 70]
Every second element : [10, 30, 50, 70, 90]
The entire list in reverse order : [100, 90, 80, 70, 60, 50, 40, 30, 20, 10]
>
```

Q4

Q4. Modifying Lists)

Create a list: colors = ["red", "blue", "green", "yellow"]

Perform the following:

Change "blue" to "purple" using indexing.

Change the last item to "black".

Print the list after each change.

Program Code:

```
# Creating a List
```

```
colors = ["red", "blue", "green", "yellow"]
```

```
# Change "blue" to "purple" using indexing.
```

```
colors[1]= "purple"
```

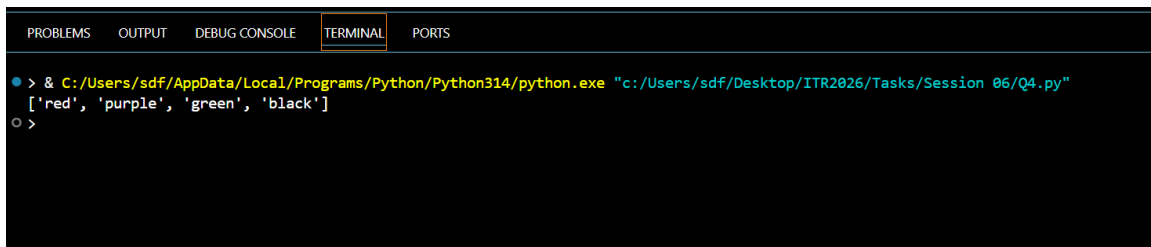
```
# Change the last item to "black".
```

```
colors[3]= "black"
```

```
# Print the list after each change.
```

```
print(colors)
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 06/Q4.py"
['red', 'purple', 'green', 'black']
>
```


Q5

Q5. (append() and insert())

Write a program that starts with an empty list.

Use append() to add 5 different city names one by one.

Use insert() to add one more city at position 2.

Print the final list.

Take user input for at least 2 cities.

Program Code:

```
# Start with an empty list
l1 = []

# Use append() to add 5 city names
l1.append("Washim")
l1.append("Akola")
l1.append("Goa")
l1.append("Amravati")
l1.append("Thane")

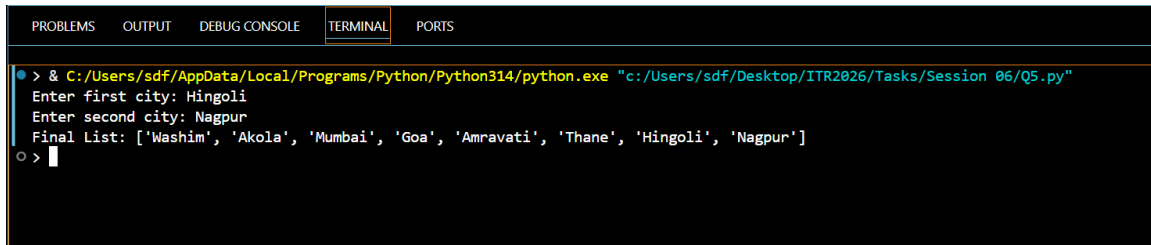
# Use insert() to add one more city at position 2
l1.insert(2, "Mumbai")

# Take user input for at least 2 cities
city1 = input("Enter first city: ")
city2 = input("Enter second city: ")

l1.append(city1)
l1.append(city2)

# Print final list
print("Final List:", l1)
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 06/Q5.py"
Enter first city: Hingoli
Enter second city: Nagpur
Final List: ['Washim', 'Akola', 'Mumbai', 'Goa', 'Amravati', 'Thane', 'Hingoli', 'Nagpur']
o > █
```

Q6

Q6. (remove(), pop(), and clear())

Create a list: items = 10, 20, 30, 20, 40, 50

Perform and print after each operation:

- a) Remove the first occurrence of 20 using remove().
- b) Remove the item at index 3 using pop() and print the removed value.
- c) Remove the last item using pop().
- d) Clear the entire list using clear().

Add comments explaining the difference between remove() and pop().

Program Code:

```
# Create a list: items = [10, 20, 30, 20, 40, 50]
items = [10, 20, 30, 20, 40, 50]

# a) Remove the first occurrence of 20 using remove().
items.remove(20)

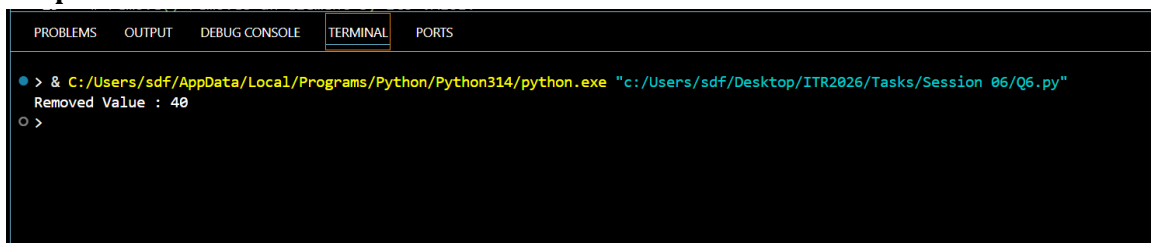
# b) Remove the item at index 3 using pop() and print the removed value.
Removed_Value=items.pop(3)
print("Removed Value :",Removed_Value)

# c) Remove the last item using pop().
items.pop()

# d) Clear the entire list using clear().
items.clear()

# Dif remove() & pop()
# remove() removes an element by its VALUE.
# pop() removes an element by its INDEX (position).
```

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 06/Q6.py"
Removed Value : 40
>
```

Q7

Q7. (index() and count())

Create a list: scores = 85, 92, 78, 92, 65, 92, 88

Find and print the index of the first occurrence of 92.

Count and print how many times 92 appears.

Take a number as input from the user and check if it exists in the list. If yes, print its index and count.

Program Code:

```
# Create a list: scores = [85, 92, 78, 92, 65, 92, 88]
scores = [85, 92, 78, 92, 65, 92, 88]

# Find and print the index of the first occurrence of 92.
indexof92=scores.index(92)
print("Index of first occurrence of 92:", indexof92)

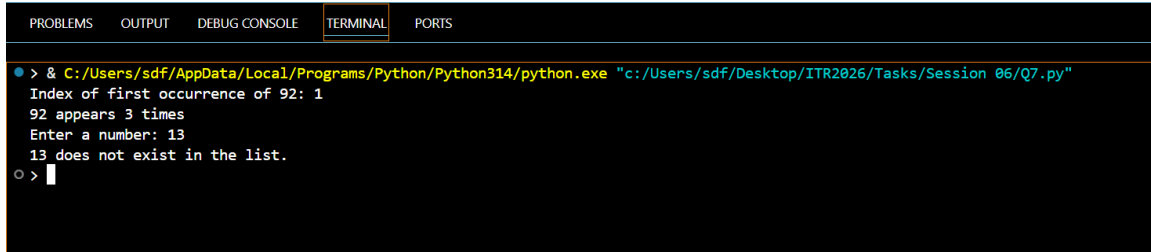
# Count and print how many times 92 appears.
count_92 = scores.count(92)
print("92 appears", count_92, "times")

# Take a number as input from the user and check if it exists in the list. If yes, print its index
and count.

# Take a number as input from the user
num = int(input("Enter a number: "))

# Check if the number exists in the list
if num in scores:
    print(num, "exists in the list.")
    print("Index:", scores.index(num))
    print("Count:", scores.count(num))
else:
    print(num, "does not exist in the list.")
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 06/Q7.py"
Index of first occurrence of 92: 1
92 appears 3 times
Enter a number: 13
13 does not exist in the list.
o >
```

Q8

Q8. (sort() and reverse())

Create a list: marks = 45, 78, 65, 90, 34, 82, 71

Session 06 (AIML) - Assignment Questions 2

Sort the list in ascending order and print it.

Sort the list in descending order and print it.

Reverse the original list (without sorting) and print it.

Add comments on the difference between sort() and reverse().

Program Code:

```
# Create a list: marks = [45, 78, 65, 90, 34, 82, 71]
marks = [45, 78, 65, 90, 34, 82, 71]

# Sort the list in ascending order and print it.
marks = [85, 92, 78, 65, 88]
sorted_list = sorted(marks)
print(sorted_list)

# Sort the list in descending order and print it.
sorted_list = sorted(marks, reverse=True)
print(sorted_list)

# Reverse the original list (without sorting) and print it.
Reversed_list=marks[::-1]
print(Reversed_list)

# Add comments on the difference between sort() and reverse().
# reverse() reverses the order of elements in the original list.

# sort() method arranges the elements of a list in ascending order by default.
# It modifies the original list and does not return a new list.
```

Output Screenshot:

```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 06/Q8.py"
[65, 78, 85, 88, 92]
[92, 88, 85, 78, 65]
[88, 65, 78, 92, 85]
```

Q9

Q9.(extend() vs append())

Create two lists:

list1 = 1, 2, 3

list2 = 4, 5, 6

Demonstrate:

a) Using extend() to add list2 to list1.

b) Using append() to add list2 to a copy of list1.

Print both results and explain the difference in comments.

Program Code:

```
# Create two lists:
```

```
list1 = [1, 2, 3]
```

```
list2 = [4, 5, 6]
```

```
# a) Using extend() to add list2 to list1.
```

```
list1.extend(list2)
```

```
print("Using extend() :",list1)
```

```
# b) Using append() to add list2 to a copy of list1
```

```
append_list = list1.copy()
```

```
append_list.append(list2)
```

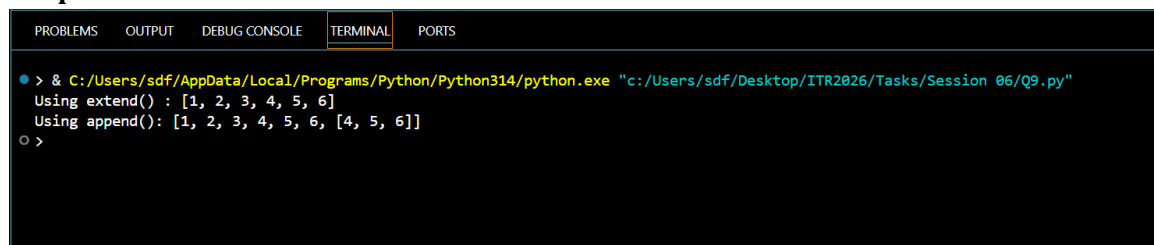
```
print("Using append():", append_list)
```

```
# Difference:
```

```
# extend() adds each element of list2 individually to the end of list1.
```

```
# append() adds list2 as a single element (nested list) to list1.
```

Output Screenshot:



```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 06/Q9.py"
Using extend() : [1, 2, 3, 4, 5, 6]
Using append(): [1, 2, 3, 4, 5, 6, [4, 5, 6]]
>
```

Q10

Q10. Mini Project - Combined Concepts)

Create a Student Marks Manager program using lists:

Take 5 subject marks as input from the user and store them in a list.

Print the list.

Add one more subject mark using `append()`.

Print the highest and lowest marks (using built-in functions).

Sort the marks in descending order.

Calculate and print the average marks.

Use `len()` to show total number of subjects.

Use proper comments, meaningful variable names, and control structures where needed.

Session 06 (AIML) - Assignment Questions 3

Program Code:

```
# Q10. (Mini Project - Combined Concepts)
```

```
# Take student details
```

```
name = input("Enter Student Name: ")
```

```
roll_no = input("Enter Student Roll No: ")
```

```
# Create empty list for marks
```

```
marks = []
```

```
# Take 5 subject marks as input
```

```
for i in range(5):
```

```
    mark = float(input(f"Enter Subject {i+1} Marks: "))
```

```
    marks.append(mark)
```

```
# Print marks list
```

```
print("\nMarks List:", marks)
```

```
# Add one more subject mark using append()
```

```
extra_mark = float(input("Enter one more subject mark: "))
```

```
marks.append(extra_mark)
```

```
print("\nUpdated Marks List:", marks)
```

```
# Print highest and lowest marks
```

```
print("Highest Marks:", max(marks))
```

```
print("Lowest Marks:", min(marks))
```

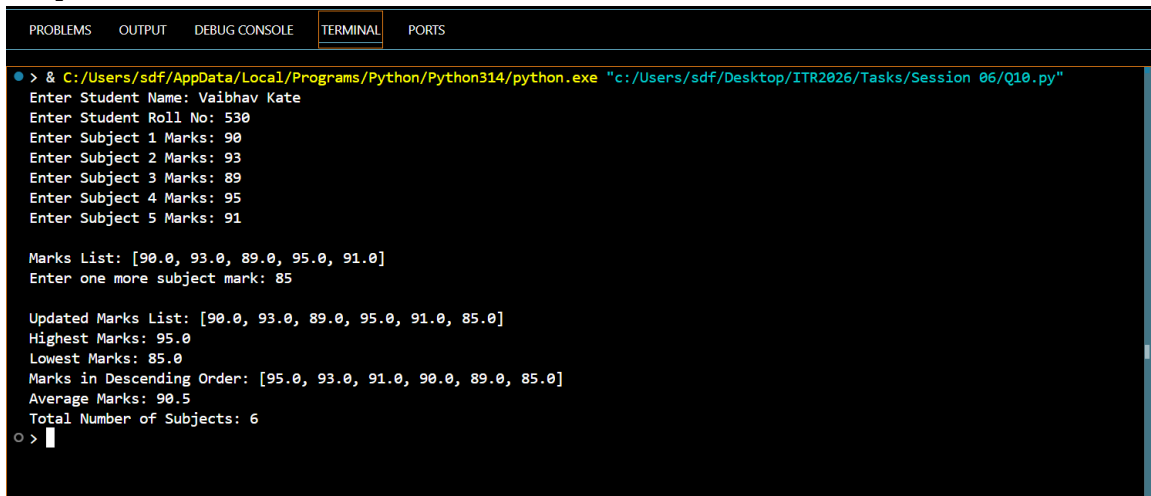


```
# Sort marks in descending order
marks.sort(reverse=True)
print("Marks in Descending Order:", marks)

# Calculate average marks
average = sum(marks) / len(marks)
print("Average Marks:", average)

# Show total number of subjects
print("Total Number of Subjects:", len(marks))
```

Output Screenshot:



```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 06/Q10.py"
Enter Student Name: Vaibhav Kate
Enter Student Roll No: 530
Enter Subject 1 Marks: 90
Enter Subject 2 Marks: 93
Enter Subject 3 Marks: 89
Enter Subject 4 Marks: 95
Enter Subject 5 Marks: 91

Marks List: [90.0, 93.0, 89.0, 95.0, 91.0]
Enter one more subject mark: 85

Updated Marks List: [90.0, 93.0, 89.0, 95.0, 91.0, 85.0]
Highest Marks: 95.0
Lowest Marks: 85.0
Marks in Descending Order: [95.0, 93.0, 91.0, 90.0, 89.0, 85.0]
Average Marks: 90.5
Total Number of Subjects: 6
>
```